Discovering and documenting system operations

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

Author of POJOs in Action and Microservices Patterns

@crichardson

chris@chrisrichardson.net

adopt.microservices.io

The starting point

0

Functional requirements

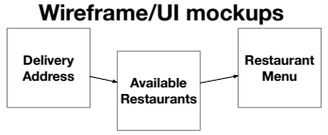
As a consumer
I want to place an Order
So that I can ….

As a Restaurant
I want to accept an Order
So that I can ….

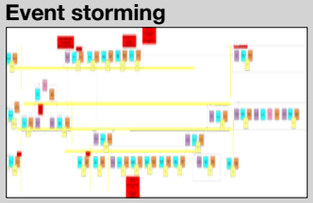
System quality attributes

- SLA: Reliability/Latency
- Scalability
- …

Wireframe/UI mockups



Event storming



The process

4 Evaluate — 5 Refactor

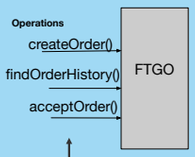
1 **Define system operations**

2 **Define subdomains**

3 **Design services and operations**

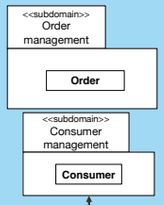
- **Group subdomains into services**
- **Design operation collaboration**

System operations



Represents an external request

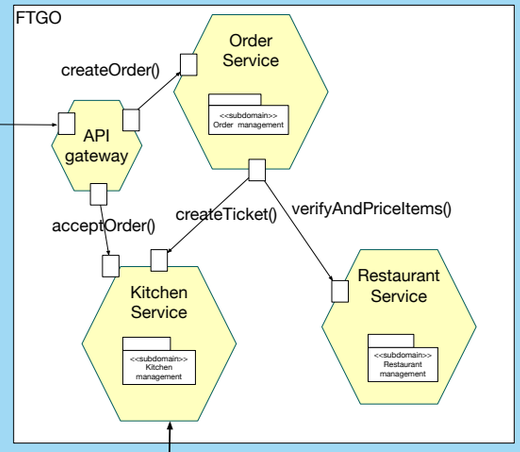
Subdomains



Models a business capability

The result

Service architecture



Service = group of subdomains

Agenda

- Overview of system operations
- Documenting system operations
- Discovering and defining commands
- Discovering and defining queries

About system operations

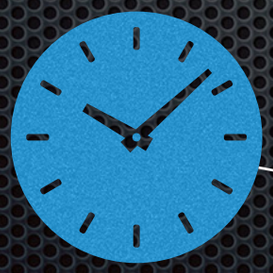
- Model application behavior
- Technology independent

Via UI

Application

Invokes

Subscribes to



Triggers

«commands»

createOrder()

cancelOrder()

…

«queries»

findOrder()

…

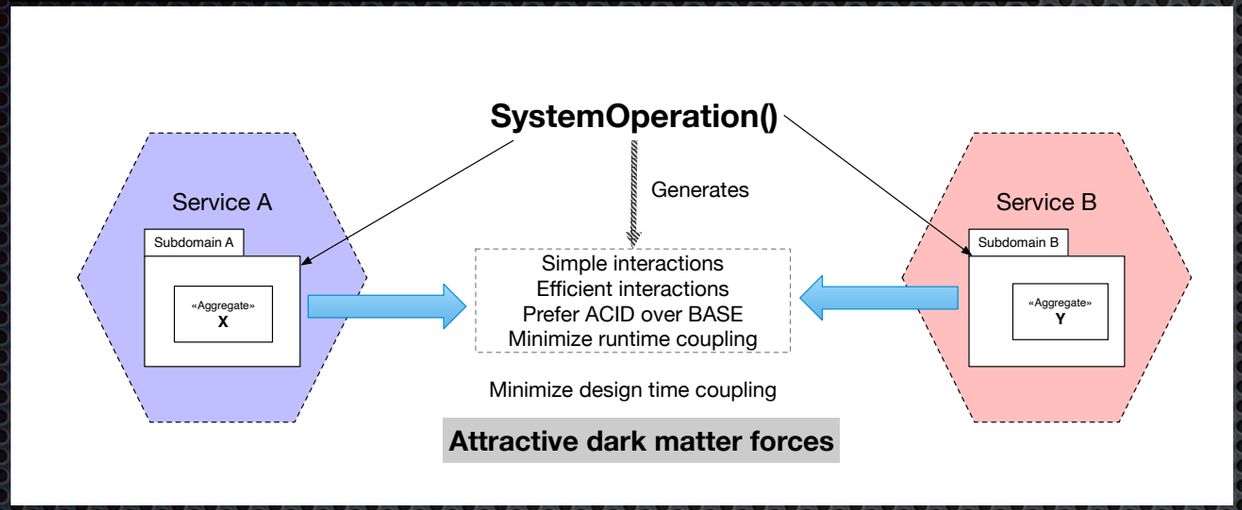
Mutates and queries

Business objects

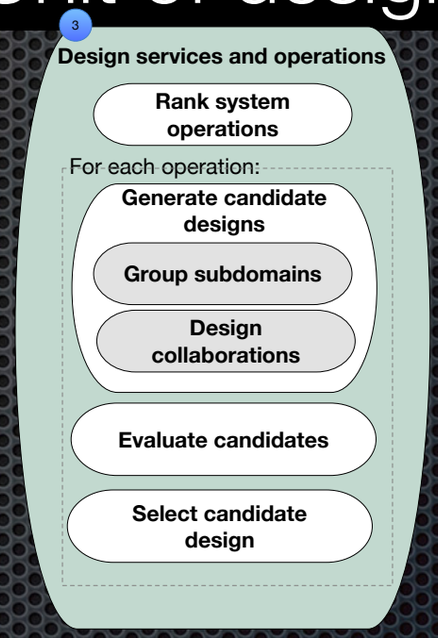
FTGO Application

About system operations

Generate dark matter forces



Unit of design



Unit of evaluation

Operation scorecard

Operation scorecard	
Name:	
Metric	Sc
Complexity	
numberOfDependencies	
numberOfSyncInvocations	
numberOfAsyncMessages	
pH	

Discovering system operations

User stories

As a consumer
I want to place an Order
So that I can ….

As a Restaurant
I want to accept an Order
So that I can ….

Event storming

Wireframe/UI mockups

System quality attributes

- SLA: Reliability/Latency
- Scalability
- …

Define system operations

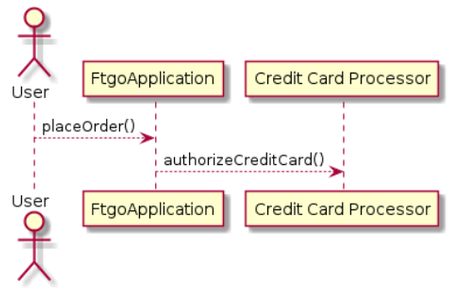
- Map stories and scenarios to commands and queries
- Map UI mockups to commands and queries
- Map Event Storming commands to system commands
- Map Event Storming read models to system queries
- Analyze legacy application

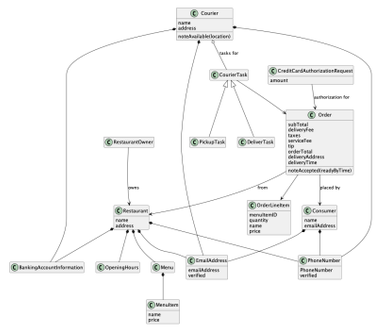
Command specification

Name:	acceptOrder()
Pre-	..
Post-	..

Query specification

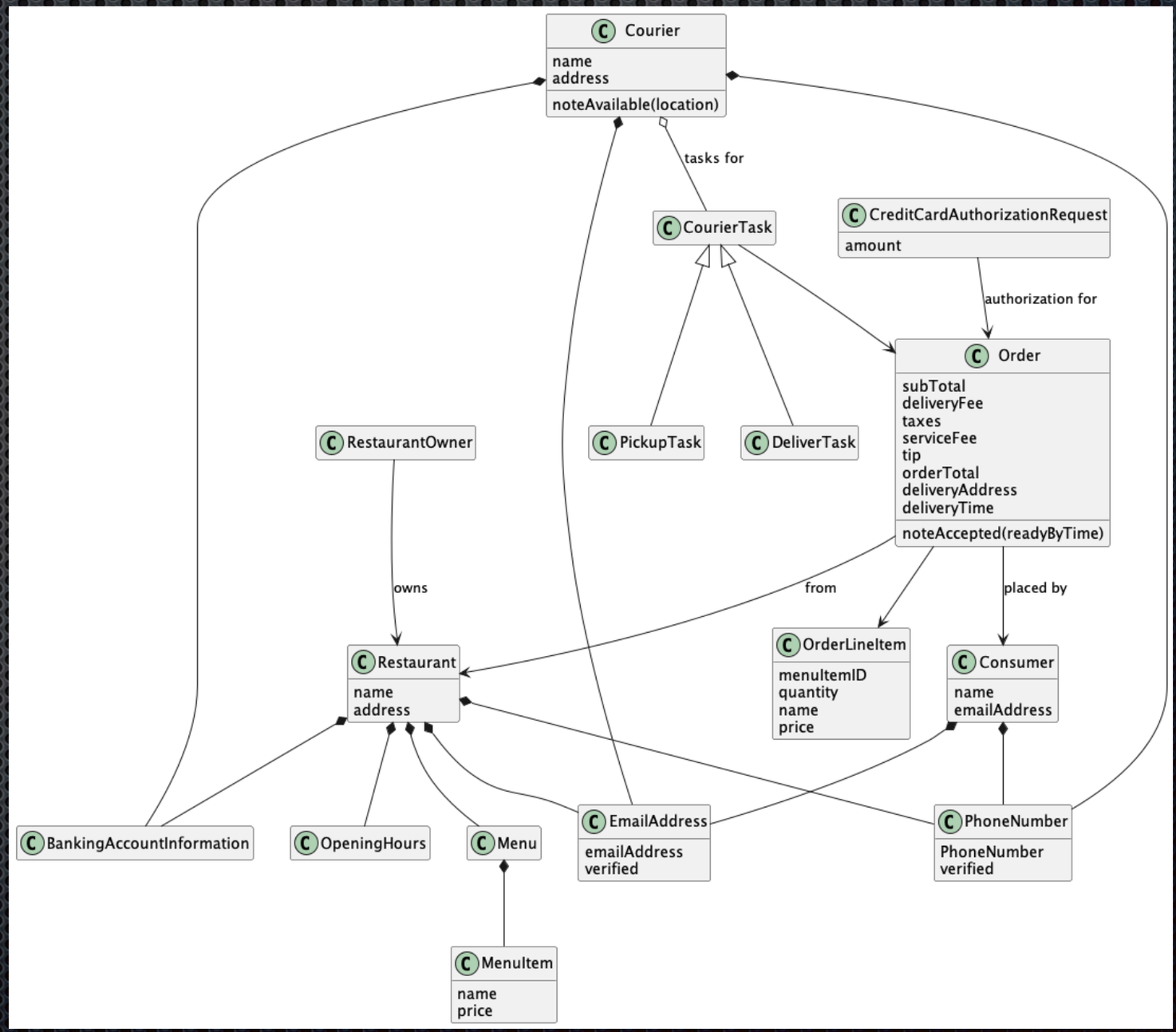
Name:	findOrderHistory
Subdomains	..
Parameters	..
Result	..





Define initial domain model while defining operations

Entities that are acted upon by system operations



Agenda

- Overview of system operations
- Documenting system operations
  - Discovering and defining commands
  - Discovering and defining queries

Documenting an operation

System command canvas	
Name	`placeOrder(consumerID, deliveryTime, deliveryAddress, restaurantID, lineItems)`
Parameters	• `consumerID` - the ID of the consumer placing the order • `deliveryTime` - the desired delivery time • `deliveryAddress` - the delivery address • `restaurantID` - the restaurant • `lineItems` - collection of `OrderLineItem(menuItemIDs, quantity)`
Return value	• `orderID`
Trigger	User command
Service Level Objectives	• Business tier: Critical, Core, User, Internal • Throughput, e.g. requests/second • Availability - ... • Latency - ... • ...
Preconditions	• a Consumer with ID `consumerID` exists • the Consumer has a valid payment method • a Restaurant with ID `restaurantID` exists • the `deliveryAddress` and `deliveryTime` are served by the restaurant • `MenuItem`'s identified by `menuItemID` exists
Postconditions	• an `Order order` was created in the `APPROVED` state • a `CreditCardAuthorization` for the `orderTotal` for the Consumer's credit card was created • `returnValue` became `order.ID`

System query canvas	
Name	findAvailableRestaurants(deliveryTime, deliveryAddress)
Parameters	• delivery address • delivery time

Return value	`select distinct r`
	`from Restaurant r`
	`where :deliveryAddress in r.serviceArea`
	`and :deliveryTime in r.openingHours`

Trigger	User command
Service Level Objectives	• Business tier: Critical, Core, User, Internal • Throughput, e.g. requests/second • Availability - ... • Latency - ... • ...

System operation: parameters and return value

User input:

- Text field
- URL values
- …

- IDs of new entities
- Updated data
- Query results

System operation: trigger

- What causes the system operation to be executed?
- User action
- External application
- Time
- …

System operation: **S**ervice **L**evel **O**bjectives (SLOs)

- Tier = degree of importance to the business, e.g.:
  - Tier 1: Critical user facing functionality
  - Tier 2: User facing functionality
  - Tier 3: Non-user facing functionality
- -ilities:….
  - Throughput, e.g. requests per second
  - Latency, .e.g 100ms p90
  - …

Key goal of documenting behavior

=

Defining relationship between operation and entities

Just enough information in order to define the service architecture

Documenting system commands: behavior - informal

- Simple describe what the command does in plain “English”
- For example: placeOrder():
  - Creates an order
  - Authorizes the credit card
  - …

Documenting system commands: behavior - formal

Pre-conditions	• orderTotal > 0 • Exists Customer c where c.ID = customerID AND c.availableCredit >= orderTotal
Post-conditions	• Order o was created • returnValue became o.ID • o.total became orderTotal • c.availableCredit became • c.availableCredit - orderTotal

Documenting system queries: describing the results

INFORMALLY:

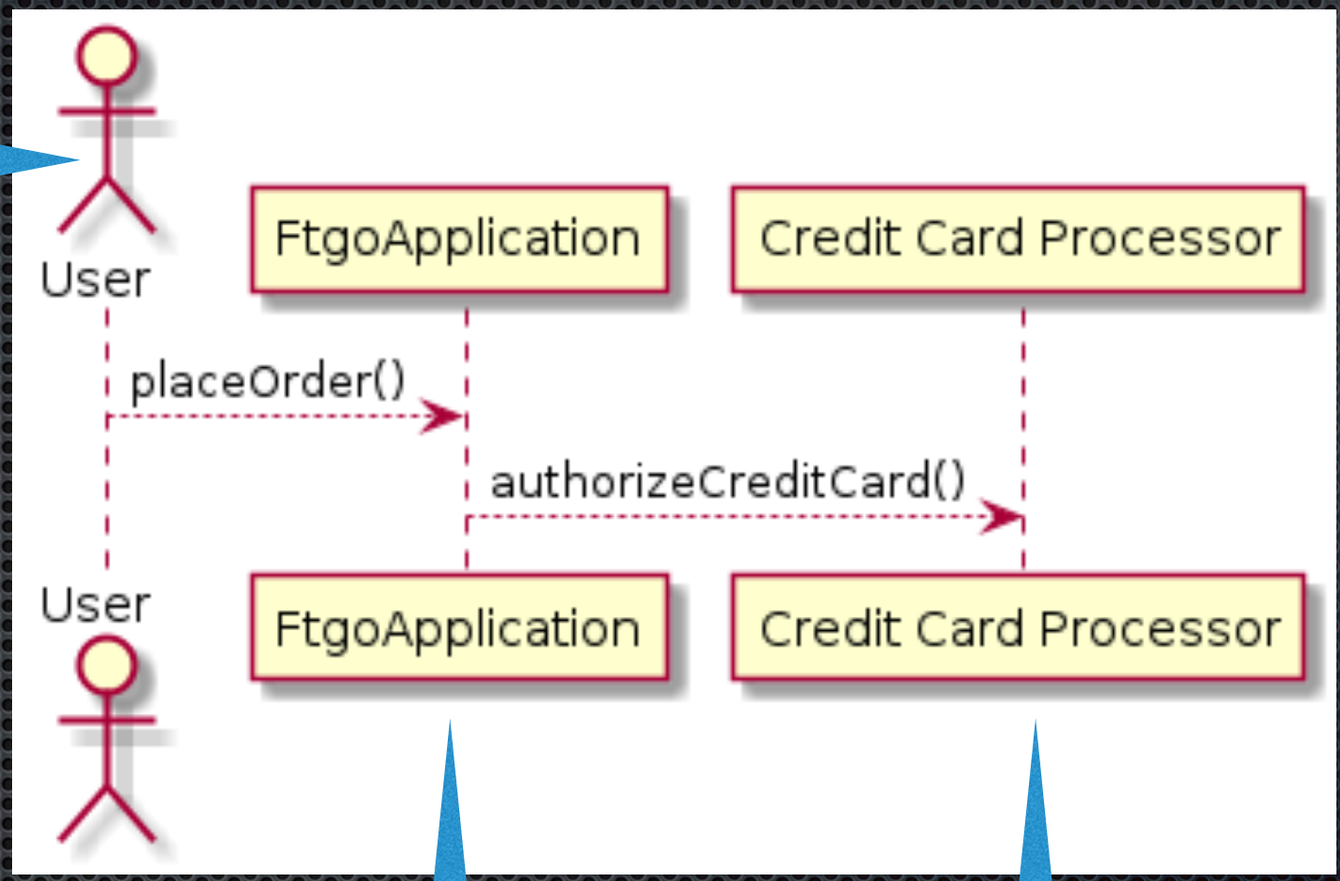
Restaurants that are
  within the service area
  AND open at the delivery time

Use JPA query language!

Return value	`select distinct r`
	`from Restaurant r`
	`where :deliveryAddress in r.serviceArea`
	`and :deliveryTime in r.openingHours`

Describe behavior: collaboration of application ⇔ actors/other applications

Actor



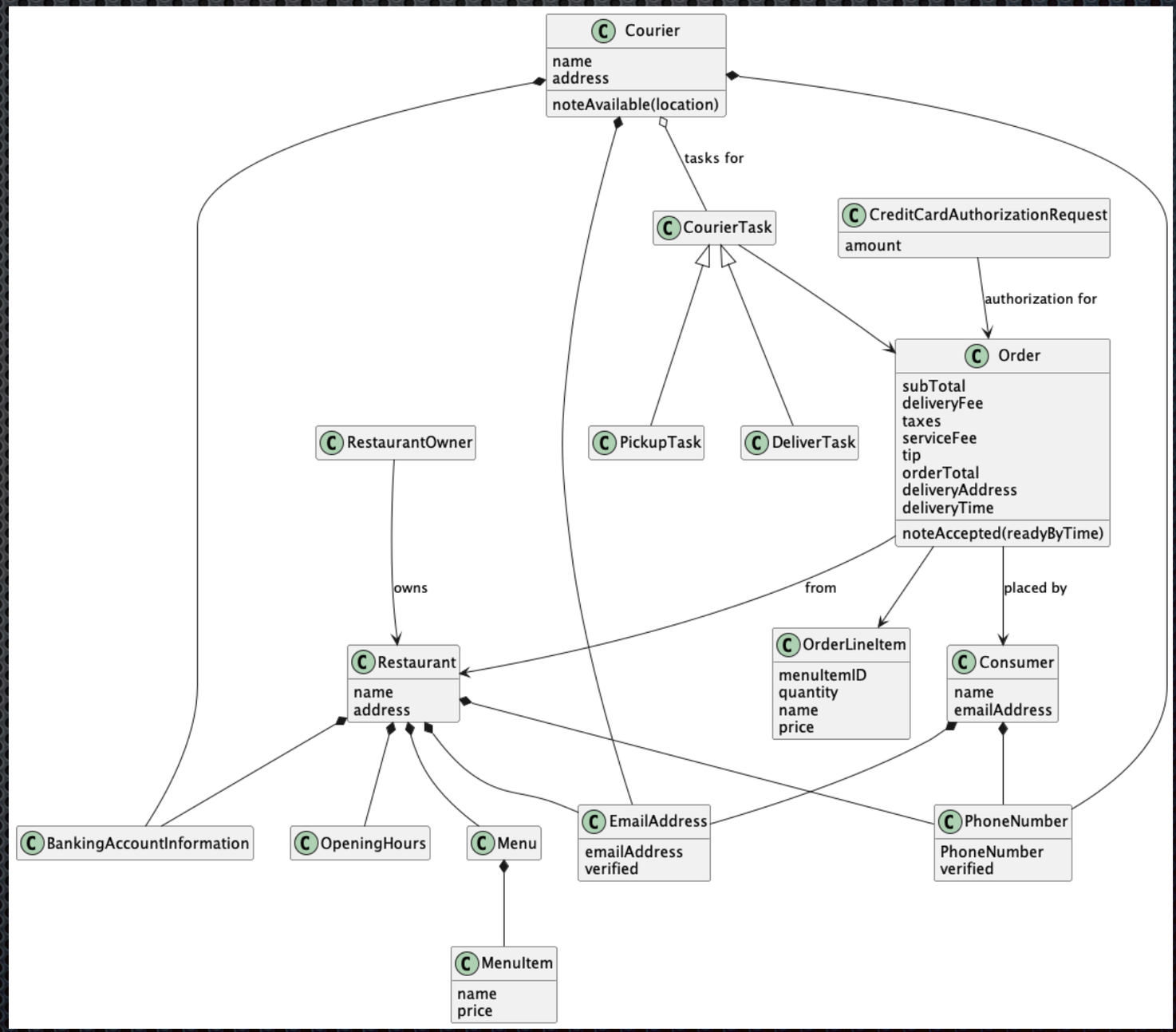
The application

External application

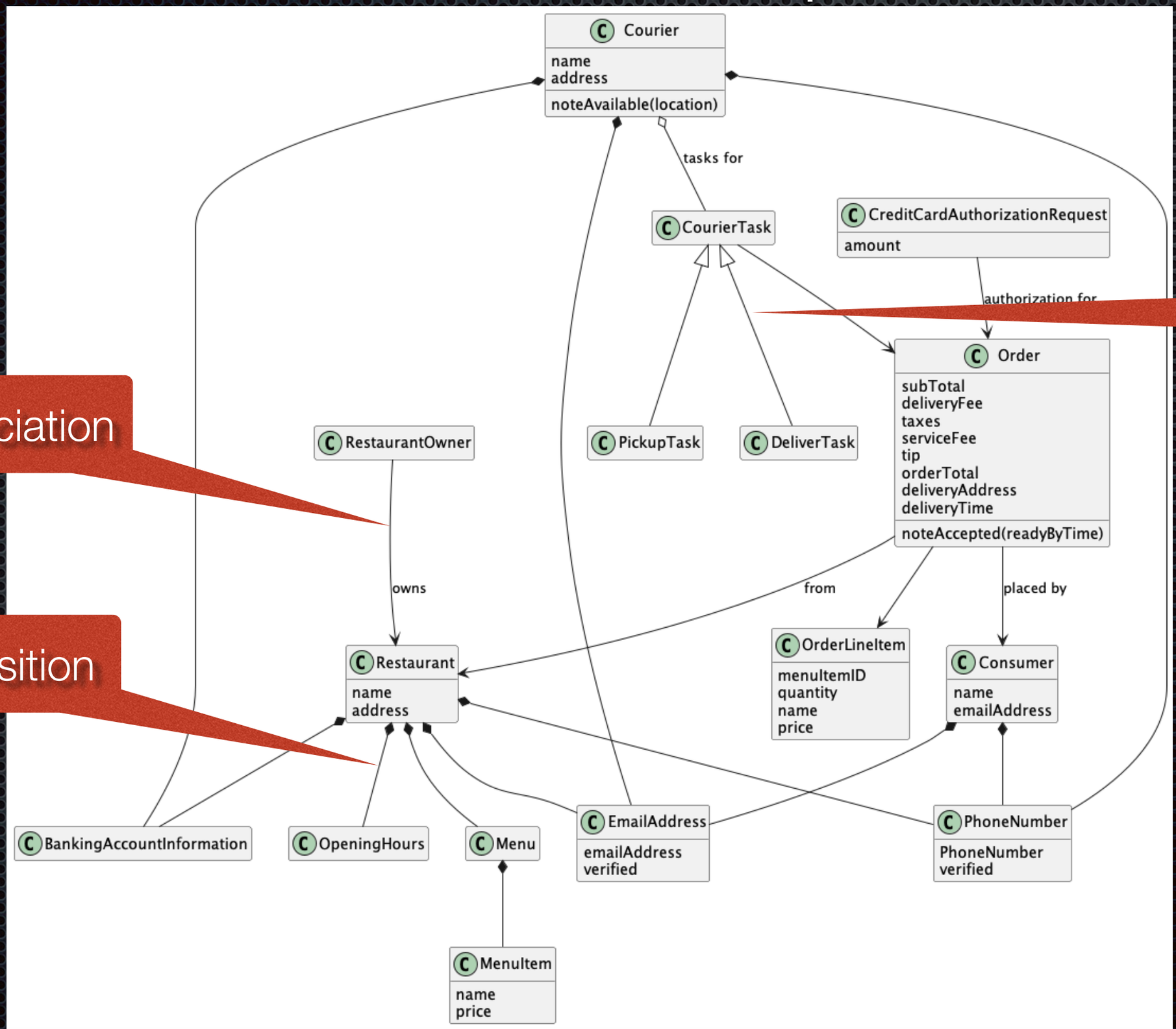
Documenting the initial domain model

Entities that are acted upon by system operations

UML class diagram: classes and their relationships



About UML class diagram: classes and their relationships



@crichardson

Agenda

- Overview of system operations
- Documenting system operations
- Discovering and defining commands
- Discovering and defining queries

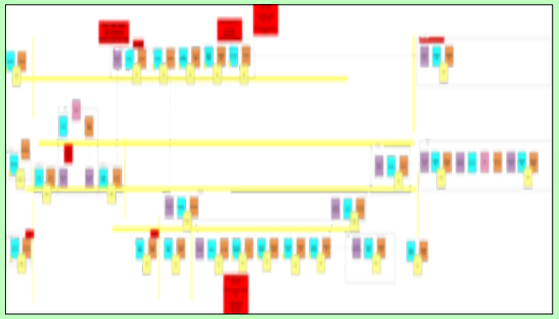
Discovering system commands

User stories

As a consumer
I want to place an Order
So that I can ….

As a Restaurant
I want to accept an Order
So that I can ….

Event storming



Wireframe/UI mockups

System quality attributes

- SLA: Reliability/Latency
- Scalability
- …

Define system commands

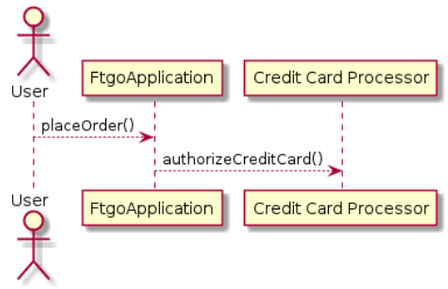
Map stories and scenarios to commands

Map UI mockups to commands

Map Event Storming commands to system commands

Command specification

Name:	acceptOrder()
Pre-	..
Post-	..



Identifying system commands from stories

Story

As a consumer
I want to place an Order
So that I can ….

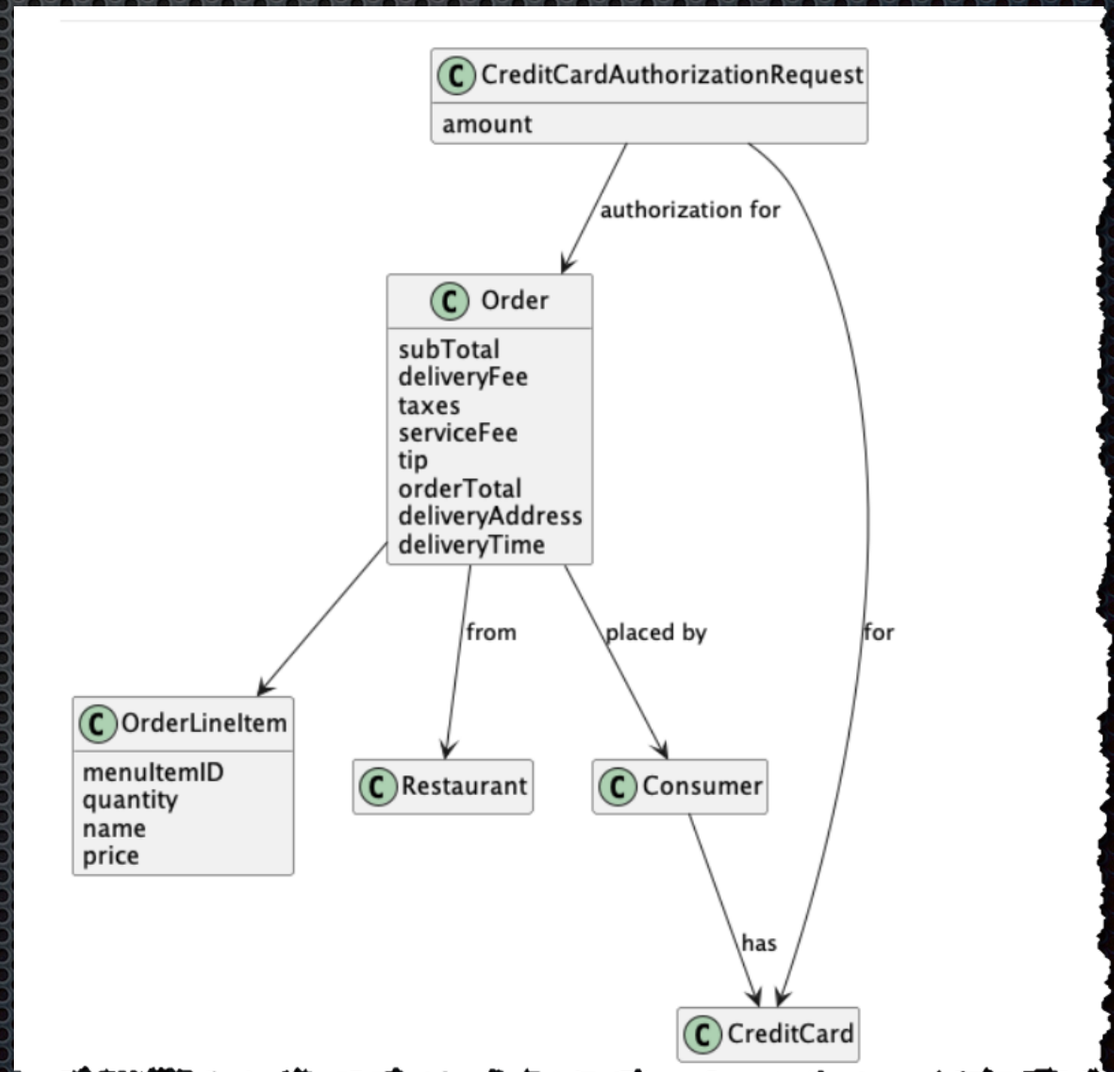
Scenario ⬇ Expand

```
Given a consumer
  And a restaurant
  And a delivery address/time that can be served by that restaurant
  And an order total that meets the restaurant's order minimum
When the consumer places an order for the restaurant
Then consumer's credit card is authorized
  And an order is created in the PENDING_ACCEPTANCE state
  And the order is associated with the consumer
  And the order is associated with the restaurant
```

Operation	`createOrder`(consumer id, payment method, delivery address, delivery time, restaurant id, order line items)
Returns	`orderId`, …
Preconditions	• The consumer exists and can place orders. • The line items correspond to the restaurant's menu items. • The delivery address and time can be serviced by the restaurant.
Post-conditions	• The consumer's credit card was authorized for the order total. • An order was created in the `PENDING_ACCEPTANCE` state.

Define the corresponding business entities

```
Given a consumer
  And a restaurant
  And a delivery address/time that can be served by that restaurant
  And an order total that meets the restaurant's order minimum
When the consumer places an order for the restaurant
Then consumer's credit card is authorized
  And an order is created in the PENDING_ACCEPTANCE state
  And the order is associated with the consumer
  And the order is associated with the restaurant
```



Identifying commands from wire flows

Shopping cart

Chicken Shawarma Wrap	$10.50
subtotal:	$10.50
Taxes:	…
Fees:	…
More fees:	…
Service charge:	…
Total:	25.95

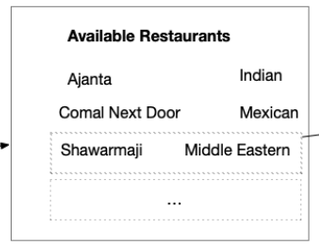
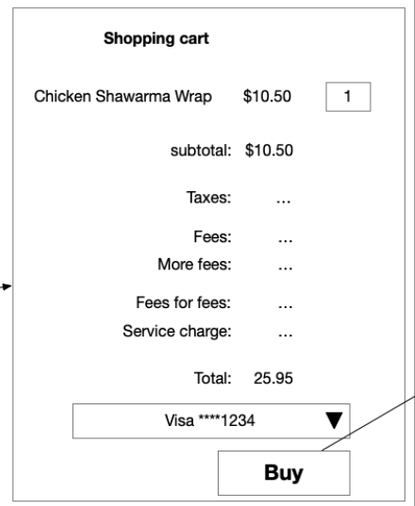
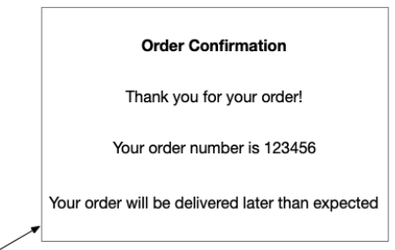
Buy

createOrder(…)

Service blueprints are helpful

Service blueprint: Create Order - system operations

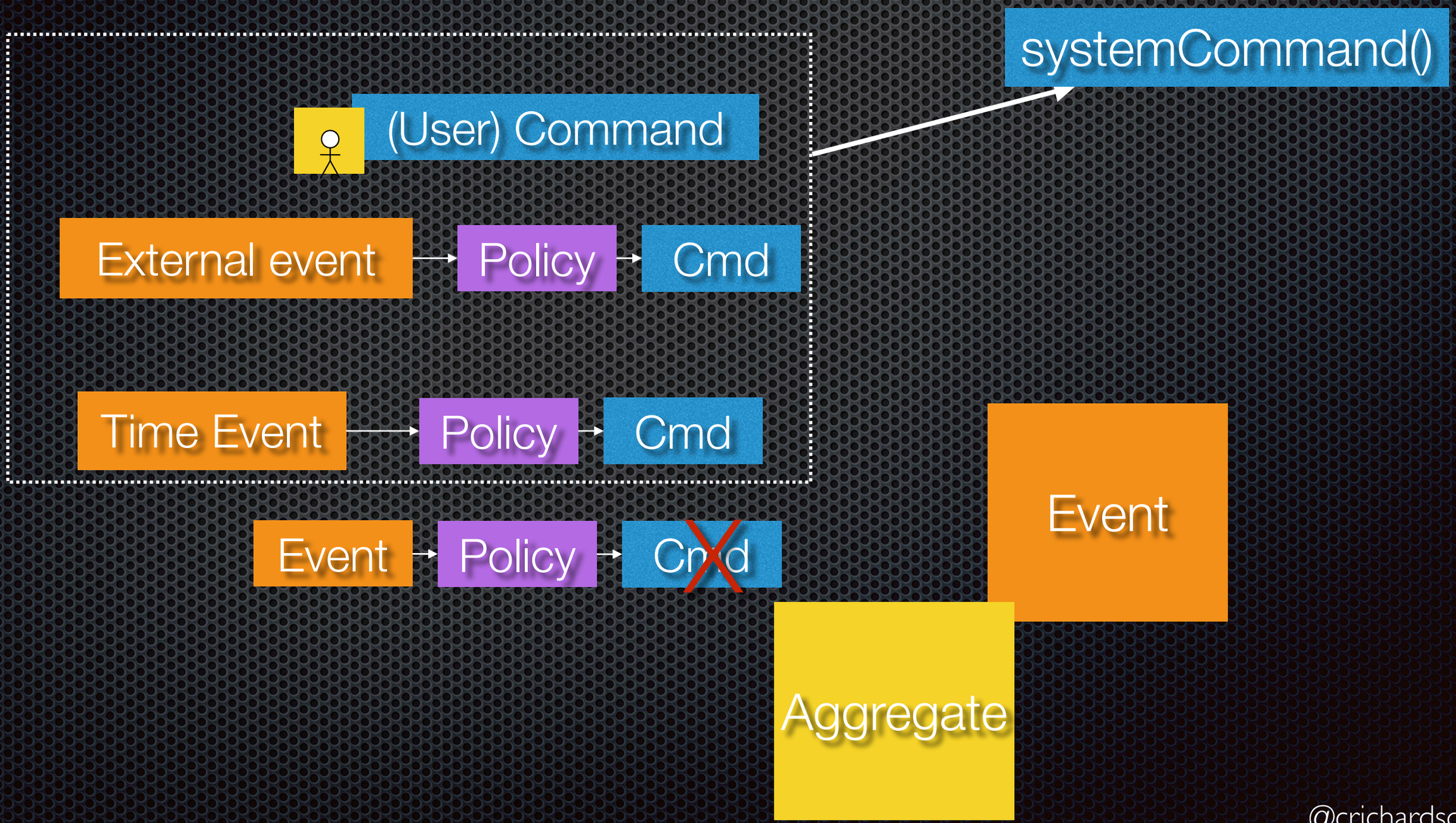
An FTGO consumer places an order for delivery

User actions	**Enter delivery address** 	**Select restaurant** 	**Choose menu items** 	**Pay** 	
System operations	updateDeliveryAddress(address, time)	updateSelectedRestaurant(restaurantId)	updateQuantities(….)	createOrder(….)	

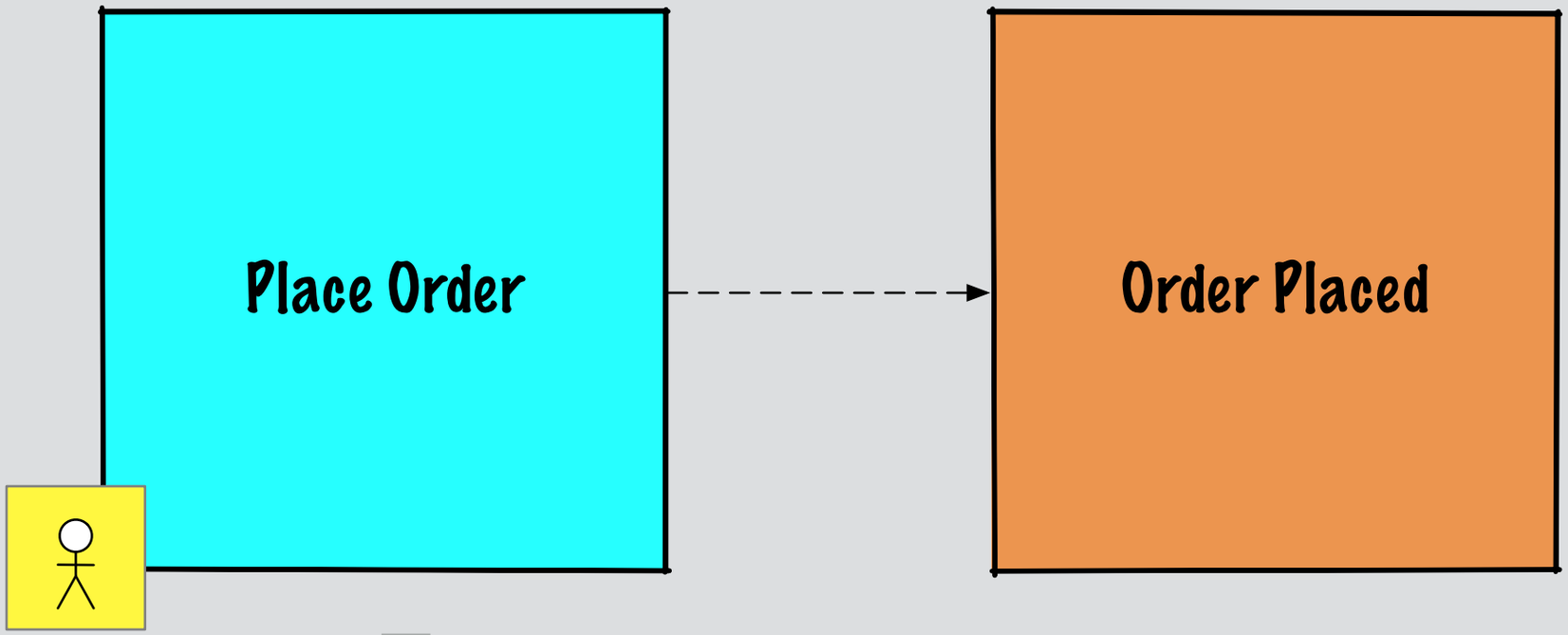
https://microservices.io/post/architecture/2023/12/07/clarifying-architecture-with-service-blueprints-service-blueprints.html

@crichardson

Deriving system commands using event storming



Defining System Operations: User invoked commands



System command: placeOrder(…)

Events => post-conditions

Postconditions	• an `Order order` was created in the `APPROVED` state • a `CreditCardAuthorization` for the `orderTotal` for the Consumer's credit card was created • `returnValue` became `order.ID`

Suggests domain model entities

Create Order

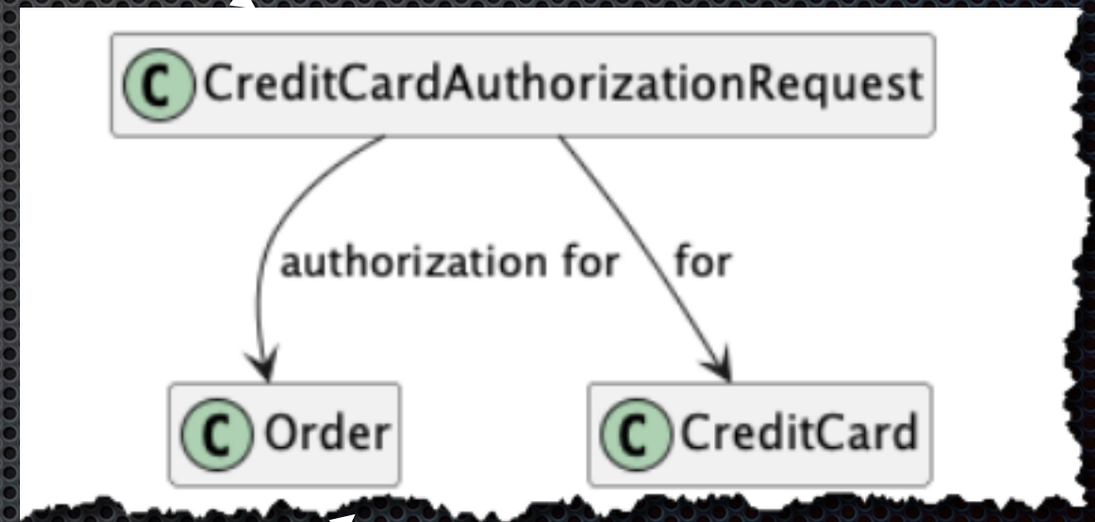
Order

Order Created

Authorize Card

Card Authorization

Card Authorized



Agenda

- Overview of system operations
- Documenting system operations
- Discovering and defining commands
- Discovering and defining queries

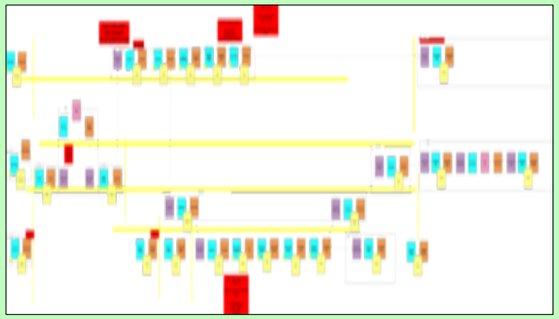
Discovering system queries

User stories

As a consumer
I want to place an Order
So that I can ….

As a Restaurant
I want to accept an Order
So that I can ….

Event storming



Wireframe/UI mockups

System quality attributes

- SLA: Reliability/Latency
- Scalability
- …

Define system operations

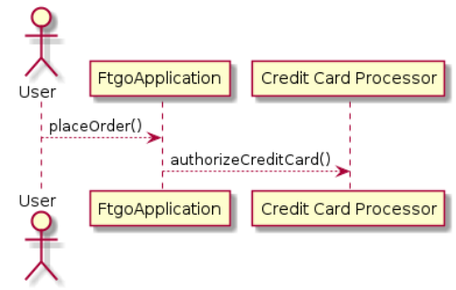
Map stories and scenarios to queries

Map UI mockups to queries

Map Event Storming read models to system queries

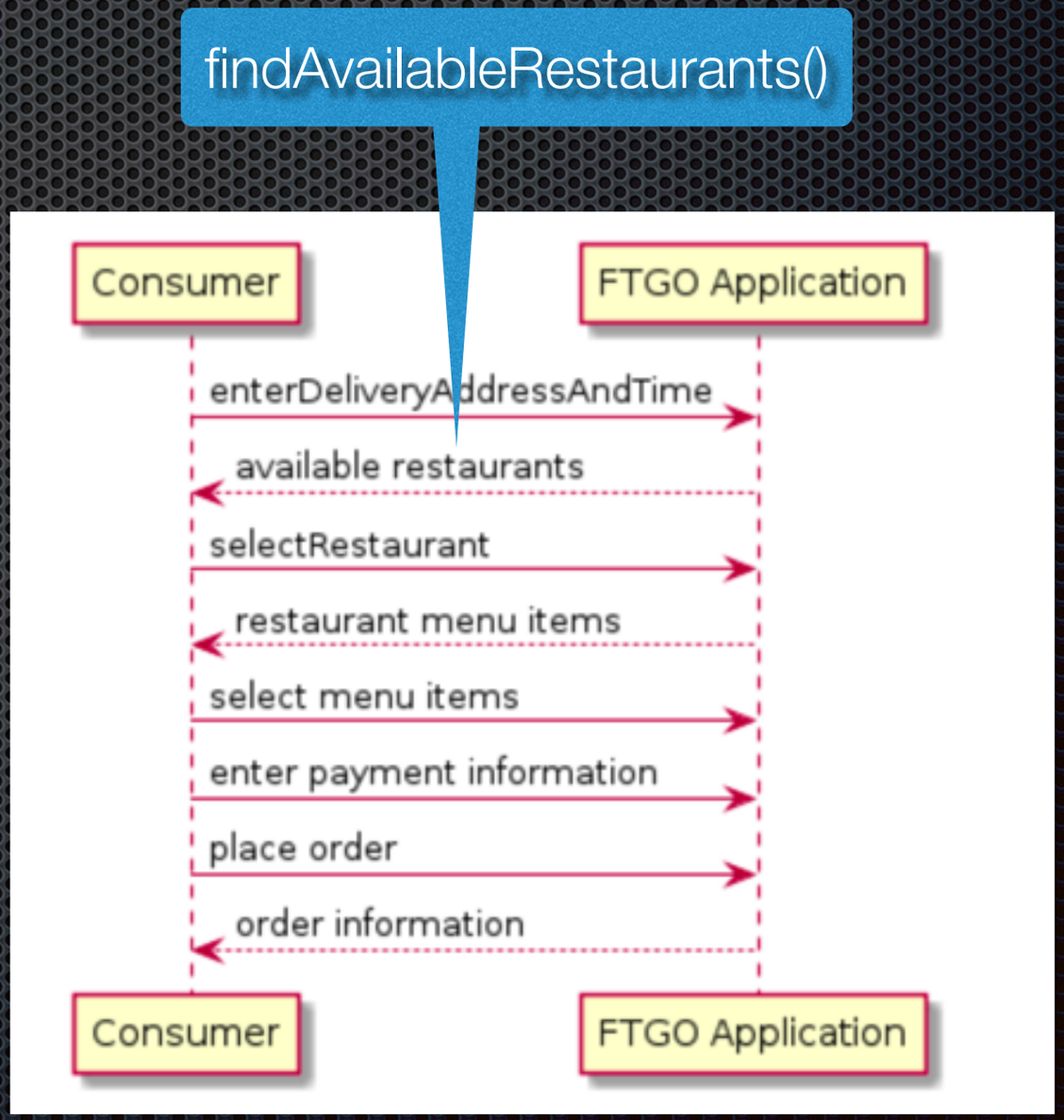
Query specification

Name:	findOrderHistory
Subdomains	..
Parameters	..
Result	..



Identifying system queries from stories

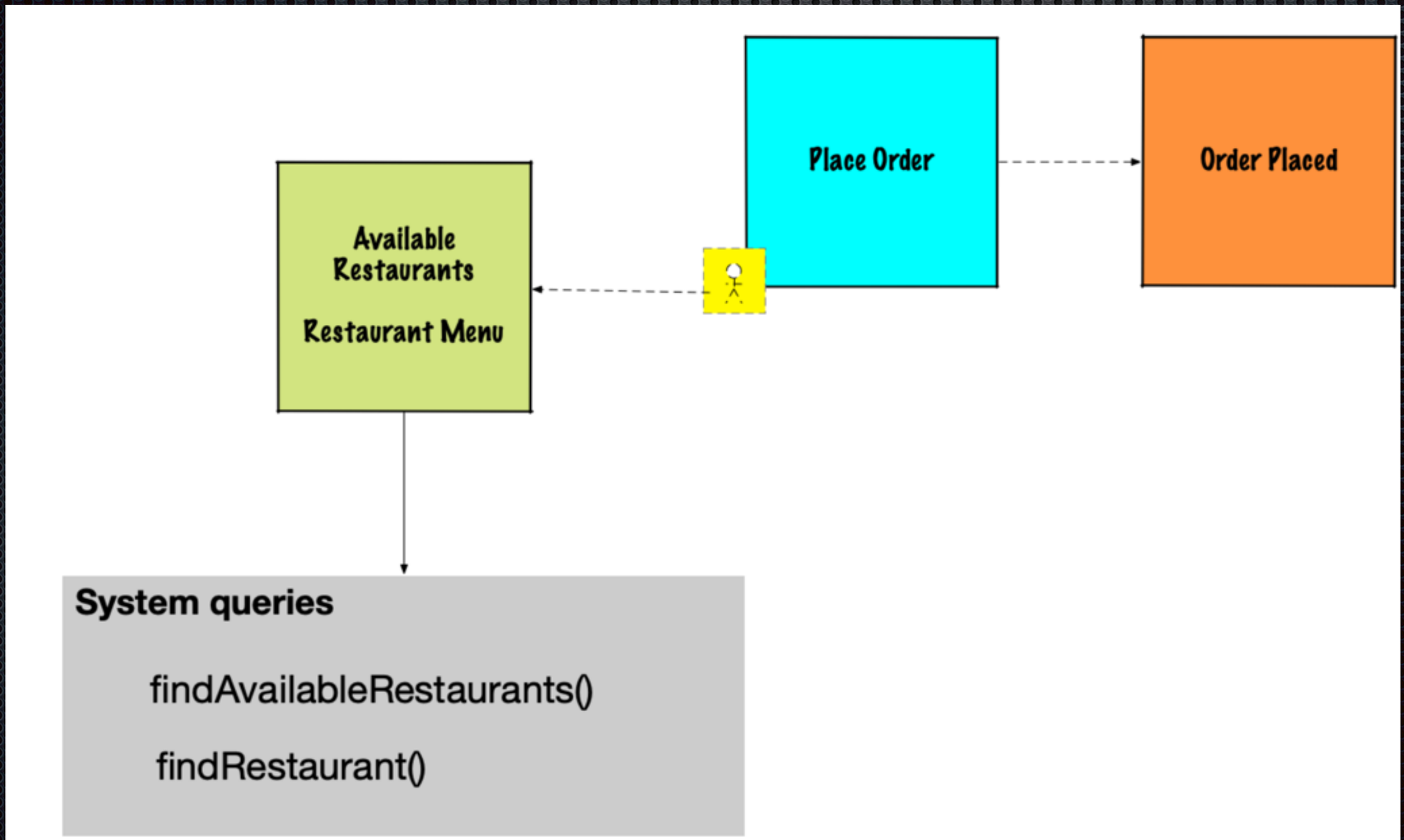
- Primarily exist to enable the UI to display information to user



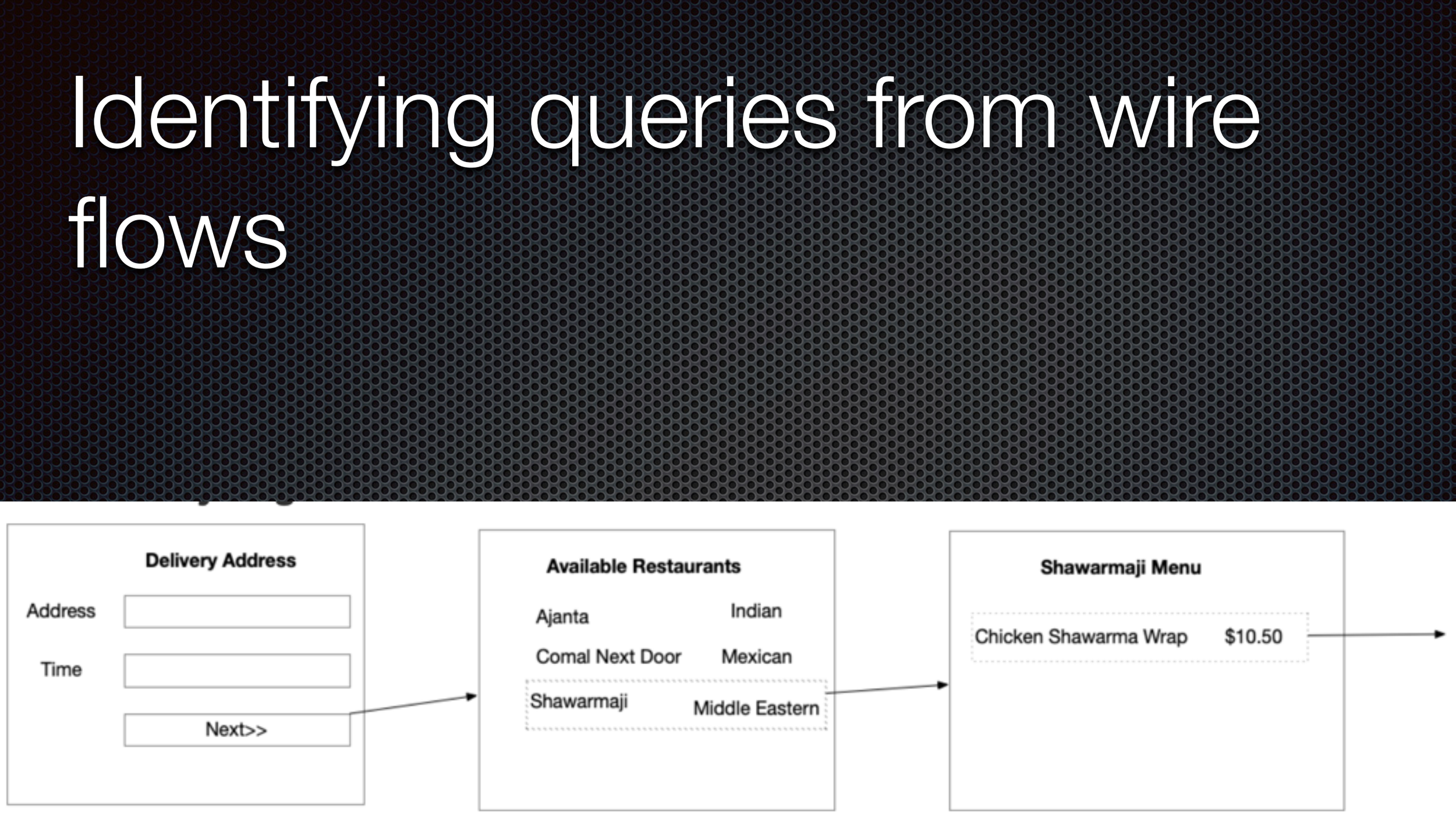
3.1.2. System query: `findAvailableRestaurants()`

Operation	`findAvailableRestaurants`(deliveryAddress, deliveryTime)
Returns	`list of available restaurants`

Event storming read models “suggest” queries



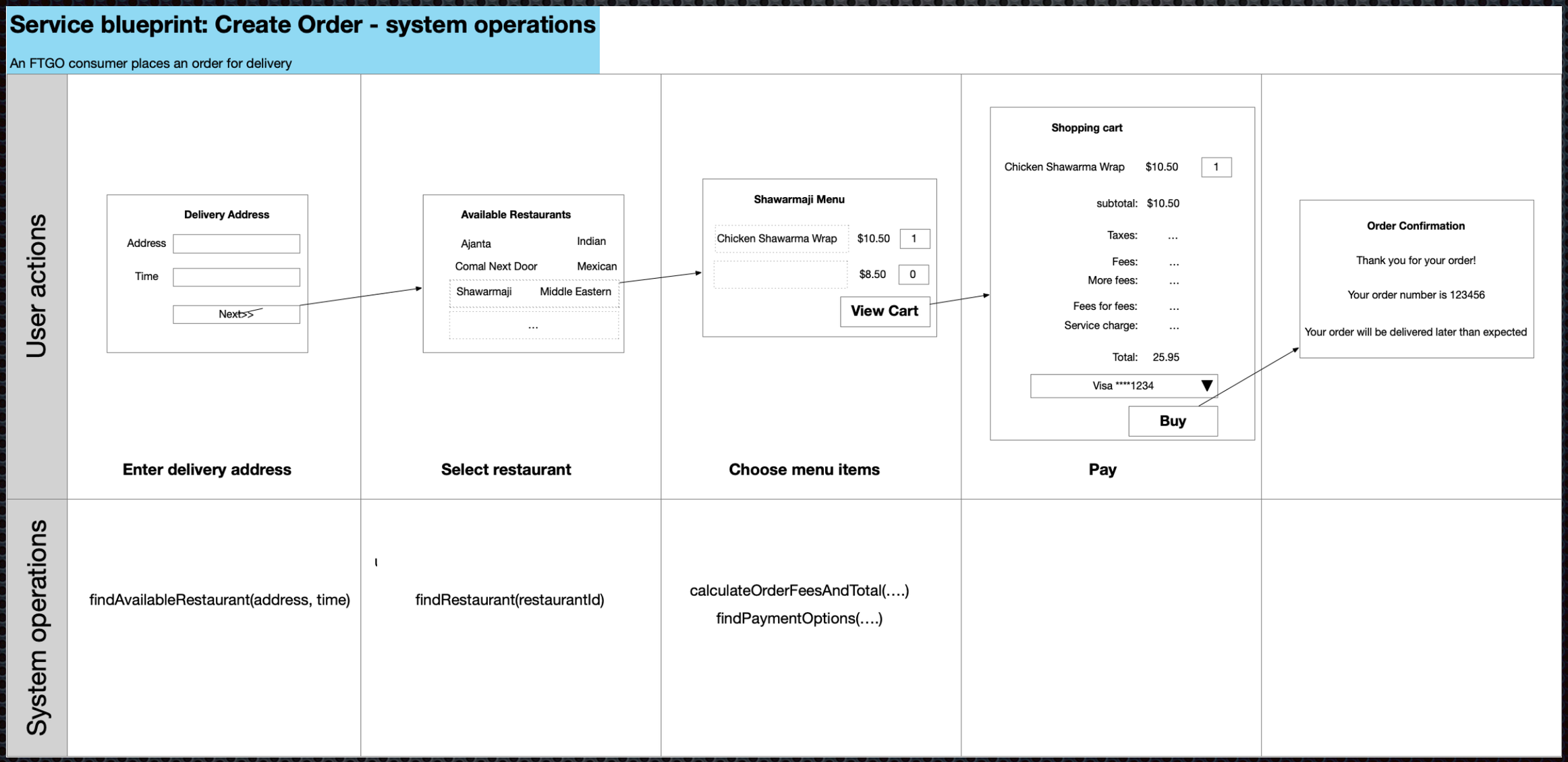
Identifying queries from wire flows



findAvailableRestaurants(…)

findRestaurant(…)

Service blueprints are helpful



https://microservices.io/post/architecture/2023/12/07/clarifying-architecture-with-service-blueprints-service-blueprints.html

@crichardson

About:

Kata #1: Discover system operations

Define and document the system operations using the templates in `01-discovering-system-operations`

- As a group:
  - Identify *first* system **command**
  - Create specification
- As a group:
  - Brainstorm system operations
  - Divide amongst pairs (or maintain a task queue)
- In pairs: document *each* assigned system operation
- As a group: Review and *harmonize* system operation definitions

`01-discovering-system-operations.pdf`

@crichardson chris@chrisrichardson.net

Questions?

adopt.microservices.io